

Agent development paradigm

AI Agent（人工智能体）的开发目前正处于爆发期。**Single Agent（单体智能体）**、**Multi Agent（多智能体/智能体集群）** 以及 **Agent + Workflow（智能体+工作流）**，涵盖了从简单应用到复杂企业级落地的主要技术路径。

以下是对这三种开发模式的**深入详细解释**，包括其核心架构、优缺点、适用场景以及技术实现细节。

1. Single Agent（单体智能体）

这是Agent开发的最基础形态，也是目前大多数C端用户接触到的形态（如ChatGPT Plus配合Plugins，或基础的AutoGPT）。

核心概念

Single Agent 依赖**单一的大语言模型（LLM）**作为“大脑”核心。它通过Prompt Engineering（提示工程）被赋予特定的人设和指令，具备**规划（Planning）**、**记忆（Memory）**和**工具使用（Tool Use）**的能力，以此来独立完成任务。

架构要素

- 大脑 (The Brain):** 负责推理和决策的LLM。
- 规划 (Planning):**
 - 子目标分解:** 将复杂任务拆解为小步骤（如 Chain of Thought）。
 - 反思与完善:** 对自己的结果进行自我批评和修正（如 ReAct 框架 - Reason + Act）。
- 记忆 (Memory):**
 - 短期记忆:** 当前的上下文窗口。
 - 长期记忆:** 通常通过向量数据库（Vector DB）实现，用于RAG（检索增强生成）。
- 工具 (Tools):** API调用能力（搜索、计算器、代码解释器等）。

工作流程

用户输入 → 意图识别 → 任务拆解 → **循环（思考 → 选择工具 → 执行 → 观察结果）** → 生成最终回答。

优缺点

- 优点:**
 - 开发成本低:** 架构简单，易于调试。
 - 响应速度较快:** 没有复杂的节点通信，延迟相对较低。
 - 通用性强:** 适合处理不需要深度专业分工的一般性任务。
- 缺点:**
 - 长程任务易失败:** 随着步骤增多，模型容易遗忘之前的指令或陷入死循环。
 - 幻觉风险:** 缺乏监督机制，一旦某一步出错，后续步骤会产生累积误差（Error Propagation）。
 - 能力瓶颈:** 一个模型很难同时精通所有领域（既是资深程序员又是法律专家）。

适用场景

- 个人助理 (Siri++) 。
- 基于文档的问答机器人 (RAG Bot) 。
- 简单的自动化任务 (如：查询天气并发送邮件) 。

2. Multi Agent (多智能体协作)

当单体Agent无法处理极高复杂度的任务时，Multi Agent System (MAS) 应运而生。这种模式模仿了人类社会的组织结构。

核心概念

Multi Agent 系统由多个拥有**不同角色 (Role)** 和**不同技能 (Skill)** 的Agent组成。它们共享任务目标，通过特定的**通信协议**进行交互，共同解决问题。

架构要素

- **角色定义 (Role Playing):** 每个Agent被设定为特定专家。例如软件开发场景中：一个产品经理Agent、一个程序员Agent、一个测试Agent。
- **通信机制 (Communication):** Agent之间如何对话。
 - **顺序式:** A做完给B, B做完给C。
 - **层级式 (Hierarchy):** 一个Boss Agent指挥多个Worker Agent。
 - **辩论/协作式:** Agent之间互相Critique (批评) 和修正。
- **环境 (Environment):** Agent们共享的状态空间 (如共享的代码库、白板) 。

工作流程 (以软件开发为例)

用户需求 → **产品经理Agent** 分析需求并生成文档 → **架构师Agent** 设计结构 → **工程师Agent** 编写代码 → **测试Agent** 运行代码并报错 → **工程师Agent** 根据报错修改 → 交付。

优缺点

- **优点:**
 - **专业度高:** 术业有专攻，每个Agent只需通过Prompt聚焦于单一职责，性能更好。
 - **自我纠错强:** 通过“多视角辩论”或“测试-修复”循环，可以大幅减少幻觉。
 - **处理复杂任务:** 能完成单体Agent无法完成的超长链路任务 (如从零写一个贪吃蛇游戏) 。
- **缺点:**
 - **成本高昂:** 多次交互意味着Token消耗成倍增加。
 - **延迟极高:** Agent之间一来一回的对话非常耗时。
 - **不可控性 (死循环):** Agent之间可能陷入无休止的争论或客套对话中，需要精细的停止机制。

代表框架

- **Microsoft AutoGen**: 允许自定义Agent交互模式。
 - **MetaGPT**: 将SOP（标准作业程序）植入Multi Agent，模拟软件公司流程。
 - **CrewAI**: 基于LangChain的角色扮演编排框架。
-

3. Agent + Workflow（智能体+工作流/SOP）

这是目前企业级应用落地的主流方向，也被称为 **Agentic Workflow（代理工作流）**。吴恩达（Andrew Ng）曾多次强调这是AI Agent的未来。

核心概念

这种模式将 **Agent的灵活性** 与 **传统工作流（Workflow）的确定性** 结合起来。它不再让Agent漫无目的地思考“我该做什么”，而是将任务固化为一个**有向无环图（DAG）**或流程图，Agent仅在特定的**决策节点**发挥作用。

架构要素

- **SOP (标准作业程序)**: 预先定义好的业务流程图。
- **确定性节点 (Deterministic Node)**: 传统的代码逻辑（如：数据库查询、格式转换、API请求），不经过LLM，保证稳定。
- **智能节点 (Agentic Node)**: 需要LLM进行意图判断、内容生成、总结归纳的节点。
- **路由/分支 (Router)**: 由Agent决定流程该走分支A还是分支B。

工作流程 (以客户售后为例)

1. **开始**: 接收用户投诉。
2. **Agent节点 (分类)**: 分析用户情绪和意图。
 - **路径A (情绪激动)**: 路由到人工客服通道。
 - **路径B (询问退款)**: 进入自动退款流程。
3. **确定性节点 (执行)**: 路径B中，系统自动查询数据库检查订单状态（不走LLM）。
4. **Agent节点 (生成)**: 根据查询结果，生成安抚话术和退款确认。
5. **结束**。

优缺点

- **优点**:
 - **可控性最强**: 限制了Agent的发挥空间，将其关在笼子里，极大降低了不可控风险。
 - **性价比高**: 不需要所有步骤都用LLM，简单的逻辑用代码完成，省钱省时。
 - **可解释性与调试**: 流程图清晰可见，哪里出错改哪里，适合企业业务集成。
- **缺点**:
 - **设计复杂度高**: 需要拥有深厚业务知识的人（Domain Expert）来设计SOP流程图。
 - **灵活性受限**: 对于流程图中未定义的新奇情况，处理能力不如纯Agent灵活。

代表工具

- **LangGraph**: LangChain推出的基于图的编排工具。
- **Dify / Coze**: 目前流行的低代码Agent编排平台，核心逻辑就是Workflow编排。

总结与对比

维度	Single Agent	Multi Agent	Agent + Workflow
核心逻辑	单兵作战，全能型	团队协作，专家分工	人机耦合，流程引导
灵活性	高	极高	中（受限于预设流程）
稳定性/可控性	低	中（依赖交互设计）	高（企业首选）
开发难度	入门级	专家级（需调优交互）	业务级（需懂业务流程）
Token成本	低	极高	中
最佳场景	简单的问答、对话	创意写作、复杂编码、头脑风暴	企业RPA、客服、审批流、数据分析